

Software Engineering & Cloud Computing course - WASP

Shubham Saha

Software Engineering Module 2 - Assignment

August 31, 2026

1 Introduction

All of today's frontier models were trained in data centers. But escalating hardware costs, limited water supply, operational and energy costs, etc., are forcing researchers to think about an alternative. A study reveals that millions of GPUs are idle and could be used to train AI models if we had the infrastructure [1]. This emerges as an idea of slowly shifting AI model training from centralized to decentralized fashion.

However, at the core of this decentralized approach, distributed peers or machines are located in entirely different parts of the world. They need to communicate over vast distances to train a single massive model. To make this happen, researchers have developed the training infrastructure, with the core focus on communication protocols, client-server management, etc. But if we zoom in a bit, we can see a research gap. Since this entire paradigm relies on global communication, the question is: how secure is the communication channel?

Currently, most research focuses on keeping the individual peers secure. For instance, frameworks like Sentinel are effective at preventing malicious clients or workers from executing gradient poisoning attacks [2]. However, by design, these frameworks assume that the channel carrying those gradients is trustworthy. So, what if an adversary attempts to perform attacks on those channels that link peers together and carry training data?

That is the question we are asking. Rather than building a defense solution right away, we are trying to study the threat space from an attacker's perspective. This will help us to build a solution that protects the entire distributed training infrastructure.

2 Lecture Principles

2.1 Testing Invariants and Metamorphic Testing

From metamorphic testing, we have learned that it is a way to test systems when we do not know the exact correct answer. The idea is that, instead of checking if an output matches a fixed number, we test how the system behaves when we change the input in a specific way. For example, if a sentiment model reads "is not" vs "isn't" then the final score shouldn't change at all. Instead of checking for a single perfect output, we are examining the relationship between two related inputs and their outputs.

This directly matches the security challenge in our research. In decentralized training, there is no oracle for what an attacked state might look like in general. An attacked run can look healthy because the loss descends normally and nothing crashes. Furthermore, distributed ML systems suffer from inherent non-determinism, in which a clean run can produce different loss curves. Therefore, we structure our experiments as metamorphic relations, which are backed by evidence invariants. First, we establish an invariant on normal behavior by running multiple clean baselines to measure the variance of the system. Then we apply channel interference on the link between peers and test the relation. Because an on-path network adversary may only have temporary access to the channel, and the attack must cause a divergence that persists even after the interference stops.

The lecture tells us that the outputs must not change under any perturbation, but in our case, the training behavior must change and remain degraded. From a defensive perspective, it makes sense to use metamorphic testing to detect violations of robustness properties, but we use it from an attacker's perspective to evaluate whether a link-level attack succeeds or not.

2.2 ML Technology Readiness Levels (ML-TRL)

From the Machine Learning Technology Readiness Level (ML-TRL) framework, we learned how an AI system moves from the theoretical first principles all the way to live deployment and continuous monitoring through ten levels. The main idea is that these levels measure the engineering progress. Each step requires more disciplined verification and validation.

Currently, decentralized training operates at levels 8 and 9, where organizations spend real money and weeks of computation to train billion-parameter models across machines spread all over the world [3]. However, the security of the communication layer connecting those machines sits at levels 1 and 2. These training infrastructures often send data over unauthenticated and unverified network channels.

Our research currently sits at level 2, where different frameworks need to be run on testbeds to demonstrate proof of principle. As mentioned before, we are not jumping straight to building defenses because we cannot defend against something that has not been characterized yet. Instead, we use a custom controlled testbed to evaluate how distributed training behaves under hostile network conditions.

Ultimately, a system is only as ready as its least mature component. So, before decentralized training can safely operate at the top levels, its communication layer has to reach the top levels as well. This is why our research focuses on level 2 testing and validation work now.

3 Guest-Lecture Principles

3.1 Requirements Elicitation for System Goals

From the guest lecture on Requirements Engineering by Julian Frattini, we got to know how to specify requirements for system goals. One good technique is to write a misuse case that describes what is not supposed to happen. Instead of just pointing out all positive features, a misuse case specifies the unwanted things, such as an unauthorized user trying to access sensitive data.

This concept defines the requirements gap in decentralized AI training infrastructure. In our literature review, we noticed that defenses only focus on misuse cases involving malicious participating peers. For example, frameworks like Sentinel are built for scenarios where a participating worker sends corrupted gradients.

However, the gap is that we have not seen any previous work that talks about misuse cases for an attacker sitting on the network link itself. There is almost no clear requirement for what must not happen if an untrusted node intercepts, delays, or tampers with training data packets in flight. Developers built the communication infrastructure for distributed training without requirements to protect against on-path tampering. Our research addresses this exact gap by modeling network-level attacks in our testbed. The plan is to define the misuse case that the communication layer needs to protect against.

3.2 The Semi-Executable Stack

The semi-executable stack shows us a diagnostic map of SE that divides software engineering into six parts. The key idea is that the lower parts run in actual machine code, but the top parts depend on human agreements, policies, and trust that no software actually checks.

This map actually explains why distributed training infrastructures lack network security. In a traditional data center, low-level communication libraries do not verify or authenticate packets in code. There, a single company owns the hardware and controls the network routes. So trust is handled by the organization rather than the software. However, decentralized training breaks this setup. It keeps the same low-level communication code with some modifications but runs it across machines on the public internet. Researchers realized the machines are untrusted, so they developed defense codes in terms of peer verification. But the path-level trust is still a gap, which leaves the communication links between those machines unverified.

To make the decentralized training infrastructure secure, path trust must also be implemented at the transport layer. Our testbed evaluates the vulnerabilities that exist while this network-level security is still missing.

4 Data Scientists, Software Engineers, and AI Engineers

4.1 What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not?

In both chapters, the author described several differences between both professions.

Data scientists focus on data preprocessing, statistical modeling, and maximizing accuracy on held-out test data in computational notebooks. They tend to have a model-centric view where they treat the machine learning (ML) model as the main component. Whereas software engineers focus on delivering complete software products. They treat ML models as unreliable functions, which might make mistakes on a larger scale. So they build safeguards, fallback logic, and user interfaces around the model to ensure the overall system remains reliable and safe.

I agree with this distinction and acknowledge it. In practice, developing a ML model requires a very different mindset than building a production system. Finding the right architecture and tuning parameters requires deep statistical knowledge. But a model with high benchmark accuracy is useless if the software crashes, leaks data, or can't handle network latency. From my perspective, I believe that both distinct roles are essential because a successful AI system requires interdisciplinary collaboration of both roles holding deep expertise.

4.2 The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area?

Chapter 2 explains that ML is usually just one component in a larger system [4], but researchers still often take an entirely model-centric view. The chapter also argues that security is a system-level property. Analyzing a machine learned model in isolation can't make any assurances about qualities like security or safety.

This actually applies to decentralized training. The key challenges between ML and non-ML components are stated explicitly. In our research, that critical interface sits between the ML training loop and the non-ML networking infrastructure. The attacks we are studying target this exact point rather than the learning algorithm itself. However, the book focuses on a distributed setting within a single organization's managed cloud. But decentralized training pushes this interface across different trust domains over the public internet. This makes the system-wide view necessary.

4.3 Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain.

My observation is that data scientists and software engineers will remain distinct roles. But they both need to learn skills of each other.

The author mentioned in the book that the unicorn who is an expert in both advanced statistical modeling and production system engineering is a myth. But it is not impossible to find. However, the fact is not about finding or preparing someone who is a unicorn, because both fields are too vast for a single role to do everything. Data scientists do need basic software engineering skills like version control, modular code writing, pipeline automation, etc., so their models can actually be deployed without creating any technical issues. Meanwhile, software engineers need ML knowledge to understand that models learn inductively and will eventually make mistakes. This will help them to design proper system-level safeguards and monitoring features. Rather than merging into one job, I think both roles must collaborate, which will help them to maintain their primary specialization while gaining enough shared knowledge to build reliable systems together.

4.4 Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct?

The term AI Engineering is still evolving and used inconsistently. In industry, people usually use this term to mean building automated pipelines, deploying models, and MLOps. To avoid this, the author prefers using phrases like "software products with machine learning components" to emphasize the entire system.

However, I think the broader idea of AI Engineering is something partly distinct. It is not any traditional software engineering, data science or standard MLOps. Because machine learning models learn from data and have no formal specifications for correctness. True AI engineering has to take a system-wide view. It is really about the challenge of connecting these imperfect models into regular software, while managing the interface between ML and non-ML components. It's also about balancing real-world system qualities like safety, latency, and security.

4.5 How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

Foundation models and RAG have changed how we build AI now. Instead of training custom models from scratch, we now connect large existing models and use prompts for further execution. This directly blurs the traditional boundaries between these roles.

Now, software engineers can build intelligent features using prompt engineering without needing a data scientist to train a model from scratch. Meanwhile, AI coding assistants automate routine implementations and reduce accidental complexity. When AI takes care of the "how", humans have to focus on the "why" and the "what" throughout the development phase. So, instead of a simple handoff where a data scientist builds a model and a software engineer deploys it, both roles now meet in the middle. They both spend their time managing prompt context, setting up agent workflows, and building system-level evaluation guardrails.

5 Paper Analysis

5.1 Paper 1 (CAIN Conference): AI Risk Assessment Frameworks and the Missing Network Layer [5]

.

1. Core ideas and their SE importance

The main idea of this paper is to map out how the software engineering community actually assesses risks in AI systems today. The authors look at 16 different AI risk-assessment frameworks coming from tech companies, governments, and research groups. They study how these frameworks handle Responsible AI principles, which parts of the software lifecycle they cover, and what practical tools they suggest using.

The work of this paper is important for AI Engineering because it points out that AI risk assessment is currently broken and disconnected. Most existing frameworks stay at a very high level with generic ethical guidelines, or they only look at narrow model problems like bias in the training data. They do not give software engineers any technical steps to assess how risks connect across the whole software system. The authors argue that we need risk frameworks that are both concrete and connected across all parts of an AI system.

2. Relation to my research

This paper relates to my research by giving clear evidence of the scope of current AI risk management. While some of the 16 frameworks do mention high-level principles like security or safety, almost all of their focus stays model-centric. They worry about data privacy, model bias, and bad predictions, but they rarely look at lower-level network threats or communication channel risks. The network is essentially treated as basic IT plumbing rather than a core AI engineering risk.

This finding supports our research topic. In decentralized AI training, machines send model updates back and forth over the public internet, making the network channel a major attack surface. This paper's study shows that existing risk frameworks largely ignore transport-level risks. This proves that the gap we are studying in decentralized training is an important issue in AI risk assessment.

3. Integration into a larger AI-intensive project

To see how this works in practice, imagine a healthcare project in which hospitals across different countries want to train a shared multimodal diagnostic medical foundation model using decentralized training. To protect patient privacy, raw patient scans stay strictly on local hospital servers, while the machines sync

model updates over the internet.

Using the paper's ideas: this project uses the C²AIRA framework to build its main AI risk assessment plan. This gives engineers and doctors a structured way to track patient privacy, clinical safety rules, and model bias at every stage of the project.

Fitting my WASP research: while the C²AIRA framework handles high-level ethical and clinical risks, our research steps in to handle the missing network layer. We use our testbed tools to check the exposure of the network channels connecting the hospitals. This makes sure the decentralized training setup is protected against packet tampering on the wire, fixing the infrastructure security gap that the high-level framework misses.

4. Adaptation of my research

In the long run, I can adapt my PhD research to build a dedicated "Transport and Communication Risk Module" that plugs directly into AI risk frameworks like C²AIRA. Right now, our research mostly runs technical attack experiments in a custom testbed to see how network tampering hurts training.

To support the bigger picture described by this paper, we can take what we learn from our experiments and turn them into concrete risk metrics and checklists. For example, we can create simple scores for node exposure, network route trust, and in-transit encryption rules. That way, before software engineers deploy a decentralized training system, they can easily evaluate and manage network-level risks alongside standard model risks.

5.2 Paper 2 (Recent SE/AI Theme): Byzantine-Robust Aggregation and In-Transit Link Vulnerabilities [6]

1. Core ideas and their SE importance

The main idea of this paper is to protect Decentralized Federated Learning (DFL) from poisoning attacks caused by malicious participating peers. In decentralized training, there is no central server managing the whole process. Instead, individual machines or nodes train their own models and share model updates directly with their neighbors over a peer-to-peer network.

To keep this training safe, the authors introduce Sentinel, a robust aggregation function. Sentinel checks incoming model updates in three simple steps: first, it uses similarity filtering to compare incoming updates against its own historical models; second, it runs bootstrap validation on a small local dataset; and third, it normalizes the weights before adding them together. This is important for AI Engineering because it allows peers a way to run collaborative learning safely, even when some participating nodes try to send bad or poisoned data.

2. Relation to my research

Sentinel connects directly to our research because it represents the state-of-the-art in what peer-level defenses can do, while also showing where they fall short. Basically, Sentinel assumes that the communication channel delivers whatever the sender transmitted without any interference. But what happens if an attacker sitting on the network link changes an honest peer's update while it travels across the wire? Sentinel will look at the modified weights, flag them as suspicious, and throw them away.

This brings up an important hypothesis for our research. Sentinel compares updates against past models. So, dropping an honest node's updates over multiple rounds might push its model completely out of sync with its neighbors. Over time, this could make even its future, clean updates look abnormal, creating a lasting exclusion where the damage continues even after the attacker leaves the network. Whether this compounding effect happens or not, Sentinel clearly proves our main point that filtering updates at the peer level is simply not enough if the network channel underneath is untrusted.

3. Integration into a larger AI-intensive project

Going back to our hypothetical medical model project:

Using the paper's ideas: each hospital runs Sentinel on its local servers. When hospitals exchange model

updates, Sentinel checks the incoming weights and blocks any corrupted or poisoned updates sent by compromised worker machines.

Fitting my WASP research: my research secures the network channels that connect these hospitals together. While Sentinel inspects the mathematical weights, our transport specifications and testbed tools make sure the packets traveling across the public internet are authenticated and never modified in transit. Together, our work makes sure the packets arrive safely on the wire, while Sentinel makes sure the data inside those packets is mathematically sound.

4. Adaptation of my research

In the long run, I can adapt my research to study how network attacks directly impact aggregation functions like Sentinel. Right now, our testbed mainly looks at overall training loss when network traffic is disrupted.

To connect directly with Sentinel, we can adapt our testbed to test our exclusion hypothesis. Specifically, we can measure the exact amount of packet tampering needed on the wire to trick Sentinel into dropping an honest peer's update. By seeing how network tampering tricks aggregation functions into treating honest machines like attackers, we can help build combined systems where network-level security checks work hand-in-hand with peer-level Byzantine defenses.

6 Evidence, Originality, and GenAI-Use Transparency

Main sources used

- Xia et al., *Towards Concrete and Connected AI Risk Assessment (C²AIRA)*, CAIN 2023 - peer-reviewed conference paper. Used for the analysis in Section 5.1.
- Feng et al., *Sentinel: An Aggregation Function to Secure Decentralized Federated Learning*, ECAI 2024 - peer-reviewed conference paper. Used for the analysis in Section 5.2 and as the reference point for peer-level defenses throughout the essay.
- Kästner, *Machine Learning in Production*, CMU, chapters 1 and 2 - open book chapters. Used for Section 4.
- Course lectures and discussion sessions (metamorphic testing, ML-TRL) and the guest lecture by Julian Frattini on requirements engineering - course material. Used for Sections 2 and 3.
- Yuan et al., *Decentralized Training of Foundation Models in Heterogeneous Environments*, NeurIPS 2022 - peer-reviewed conference paper. Used for the GPU underutilization claim in Section 1.
- Folding@home, *OS Statistics* - project statistics page, not a study. Used only for the volunteer computing device count in Section 1.
- Prime Intellect Team, *INTELLECT-2: A Reasoning Model Trained Through Globally Decentralized Reinforcement Learning*, arXiv 2025 - preprint/industry technical report. Used for the scale of deployed decentralized training in Section 2.2.
- Mohaghegh Dolatabadi et al., *Sentinel: Stagewise Integrity Verification for Pipeline Parallel Decentralized Training*, arXiv 2026 - preprint, not yet peer reviewed. Used in Section 1. Note this is a different paper from the ECAI 2024 Sentinel discussed in Section 5.2.

GenAI use

I used GenAI tools for fact-checking the references, outlining things I wrote, and language checking. I hereby confirm that the analysis and wording are my own.

References

- [1] Binhang Yuan, Yongjun He, Jared Quincy Davis, Tianyi Zhang, Tri Dao, Beidi Chen, Percy Liang, Christopher Ré, and Ce Zhang. Decentralized Training of Foundation Models in Heterogeneous Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2206.01288>.

- [2] Hadi Mohaghegh Dolatabadi, Thalaiyasingam Ajanthan, Sameera Ramasinghe, Chamin P. Hewa Koneputugodage, Gil Avraham, Yan Zuo, Violetta Shevchenko, and Alexander Long. Sentinel: Stagewise Integrity Verification for Pipeline Parallel Decentralized Training, March 2026. URL <https://arxiv.org/abs/2603.03592>. Pluralis Research.
- [3] Prime Intellect Team. INTELLECT-2: A Reasoning Model Trained Through Globally Decentralized Reinforcement Learning, May 2025. URL <https://arxiv.org/abs/2505.07291>.
- [4] Christian Kästner. From Models to Systems. In *Machine Learning in Production*. Carnegie Mellon University. URL <https://mlip-cmu.github.io/book/02-from-models-to-systems.html>.
- [5] Boming Xia, Qinghua Lu, Harsha Perera, Liming Zhu, Zhenchang Xing, Yue Liu, and Jon Whittle. Towards Concrete and Connected AI Risk Assessment (C²AIRA): A Systematic Mapping Study. In *Proc. 2023 IEEE/ACM 2nd Int. Conf. on AI Engineering – Software Engineering for AI (CAIN)*, pages 104–116. IEEE, 2023. doi: 10.1109/CAIN58948.2023.00027.
- [6] Chao Feng, Alberto Huertas Celdrán, Janosch Baltensperger, Enrique Tomás Martínez Beltrán, Pedro Miguel Sánchez Sánchez, G  r  me Bovet, and Burkhard Stiller. Sentinel: An Aggregation Function to Secure Decentralized Federated Learning. In *Proc. 27th European Conf. on Artificial Intelligence (ECAI)*, pages 1760–1767. IOS Press, 2024. doi: 10.3233/FAIA240686.